

Промежуточный отчет по программному проекту

1. Основные планы и этапы проекта

1.1 Краткое описание проекта: Проект предусматривает создание Telegram-бота для интеграции с существующими платформами через вебхуки. Бот будет автоматически извлекать данные и предоставлять пользователю доступ к нужной информации, выдавать отчеты и изменения.

Название проекта: Разработка Telegram-бота для интеграции с информационными системами

Цель проекта: Разработать Telegram-бота для получения информации из внешних информационных систем.

Краткое описание задач:

1. Изучить набор систем с API в программе CheckOffice.
2. Настроить взаимодействие бота с платформой через вебхуки.
3. Реализовать функциональность извлечения данных.
4. Создать удобный интерфейс Telegram-бота для пользователей.

1.2 Планы и этапы выполнения проекта:

Этап проекта	Описание работ	Ожидаемые результаты	Сроки выполнения
Выявление требований заказчика	Выявление функциональных требований	Согласованный с обеих сторон список полных	01.12.2025

		требований к проекту	
Отчуждение данных	Получение доступа к системам и базам данных	API ключи к информационной системе CheckOffice и к базе данных с логинами и паролями пользователей и документация к системе	04.12.2025
Изучение набора предоставленных систем	Изучение структуры информационной системы	Информация о структуре информационной системы	12.12.2025
Написание Технического задания	Написание Технического задания на основе выявленных требований согласно ГОСТу.	Готовое Техническое задание, согласованное всеми сторонами.	14.12.2025
Разработка API для интеграции	Подключение к API CheckOffice. Настройка получения данных через вебхуки. Создание запросов для получения данных из системы.	API, который будет использоваться для предоставления данных Telegram-боту	01.03.2026
Создание Telegram-бота	Настройка входа. Создание интерфейса. Настройка получения данных от созданного API.	Telegram-бот, который взаимодействует с платформой через вебхуки	01.03.2026

	Разработка формата вывода данных в Telegram-боте и дополнительной информации для вывода пользователю.	и присылает данные пользователю по запросу.	
Испытания программы	Разработка, согласование и утверждение порядка и методики испытаний. Корректировка программы и программной документации по результатам испытаний.	Проведено тестирование. Устранены ошибки и недочеты.	10.04.2026
Внедрение	Подготовка и передача программы.	Программа готова к использованию заказчиком. Загрузка программы и программной документации в Smart LMS	20.04.2026

2. Используемый технологический стек и его обоснование

2.1 Перечень используемых технологий:

Технология/Инструмент	Описание	Причины выбора
Язык программирования: Python	Python — интерпретируемый язык высокого уровня с лаконичным	Большое количество готовых библиотек для работы с Telegram Bot API,

	синтаксисом и обширной стандартной библиотекой.	HTTP-запросами, базами данных и т.д. Поддержка асинхронного программирования (asyncio), может обслуживать множество запросов одновременно.
Фреймворк для Telegram-бота: aiogram	Асинхронный Python-фреймворк для разработки Telegram-ботов, построенный поверх asyncio.	Полноценная поддержка вебхуков, что важно для интеграции с внешними системами и для минимизации задержек.
Серверная часть: FastAPI	Веб-фреймворк для Python, предназначенный для создания API.	Высокая производительность, поддержка асинхронности, удобная работа с запросами и ответами.
HTTPX	Библиотека для выполнения HTTP-запросов в Python с поддержкой асинхронного режима работы.	Выбрана как современное и надежное средство для реализации HTTP-взаимодействия, так как поддерживает асинхронную модель работы, позволяет

		эффективно обрабатывать сетевые запросы без блокировки сервера и хорошо интегрируется с асинхронной архитектурой FastAPI.
Система контроля версий: Git	Git используется для версионного контроля исходного кода Telegram-бота и всей связанной логики.	Возможность отслеживать изменения в коде и при необходимости откатиться к предыдущим версиям. Также удобен для командной работы.

2.2 Обоснование выбранного технологического стека:

Использование языка Python помогает быстро реализовать прототип бота, интегрировать его с внешними API и гибко расширять функциональность (новые команды, новые отчеты, дополнительные системы).

Фреймворк для Telegram-бота: aiogram упрощает реализацию диалоговой логики бота, подключение вебхуков и асинхронное обращение к внешним API. Это напрямую помогает реализовать функциональность получения информации из информационных систем и предоставления отчетов пользователю.

Для реализации серверной части проекта был выбран стек FastAPI и HTTPX, так как он наилучшим образом соответствует целям разработки Telegram-бота для получения информации из внешних информационных систем. FastAPI позволяет создать надежное и структурированное API, через которое осуществляется

взаимодействие между ботом и внешними системами. Он удобен для приёма вебхуков, обработки входящих запросов и формирования ответов в едином формате, что напрямую связано с задачами интеграции и извлечения данных. Асинхронная модель FastAPI обеспечивает одновременную обработку нескольких запросов без потери производительности, что особенно важно при работе с внешними API и большом количестве пользовательских обращений.

Библиотека HTTPX дополняет данный стек и используется для обращения к внешним информационным системам по HTTP. Она поддерживает асинхронное выполнение запросов и хорошо интегрируется с FastAPI, благодаря чему снижается время ожидания ответов от внешних сервисов и повышается общая устойчивость системы. Использование HTTPX позволяет эффективно получать данные из API и передавать их в Telegram-бот без блокировки серверной части.

3. Критерии оценивания проекта

В зависимости от курса и уровня студента, сформируйте, пожалуйста, перечень количественных критериев (не менее 3 критериев), которыми будет оцениваться проект. Вы можете предложить и обосновать свои критерии, либо использовать такие критерии как, например:

Критерий	Описание
Использование инструмента автоматической документации API	Будут использованы
Использование адаптивного дизайна	Не будут использованы
Работа с базой данных	Будут использованы
Управление доступом	Будут использованы

Функциональность - Процент выполнения функциональных требований	Выполненные требования в процентах от общего количества
Функциональность - Количество реализованных функций	Абсолютное количество функций, которые работают правильно
Производительность и эффективность - Время отклика	Среднее время ответа программы на запросы пользователя (в секундах)
Производительность и эффективность - Потребление ресурсов	Среднее потребление памяти (в МБ) и процессорного времени (в %)
Качество кода - Количество обнаруженных ошибок (bugs/KLOC)	Количество ошибок на 1000 строк кода
Документация и оформление - Полнота документации (%)	Процент от требуемого объема документации
Тестирование - Количество написанных тестов	Общее количество юнит-тестов, интеграционных тестов и других видов тестов
Тестирование - Процент успешных тестов (%)	Процент успешно пройденных тестов из общего количества
Совместимость и кроссплатформенность - Количество интеграций	Число успешных интеграций с другими системами или сервисами
Соблюдение сроков и плана - Процент выполнения работы в срок (%)	Процент задач, выполненных в срок
Соблюдение сроков и плана - Количество дней отклонения от плана	Общее число дней отклонения от плана
Использование технологического стека - Процент использования функциональности стека (%)	Процент использования функциональности выбранного стека технологий

Оценка командной работы - Среднее время коммуникации (в часах)	Среднее время, потраченное на обсуждение задач и решение вопросов
Оценка командной работы - Количество завершенных задач на каждого участника	Общее число задач, выполненных каждым членом команды

4. Особые пометки

Отсутствуют.